

Lecture 5

Simple Perceptron

Elena Raponi | Neural Computing course



Universiteit
Leiden
The Netherlands

Discover the world at Leiden University

Index

1. Hebbian Learning
2. Definition of Simple Perceptron
3. Network Topologies
4. Activation Functions
5. Learning Rule
6. Binary Classification
7. Perceptron's optimality
8. Proof of Convergence of the Learning Rule
9. Multi-layer Perceptron
10. Gradient Descent
11. Intro to Back-propagation



Hebbian Learning

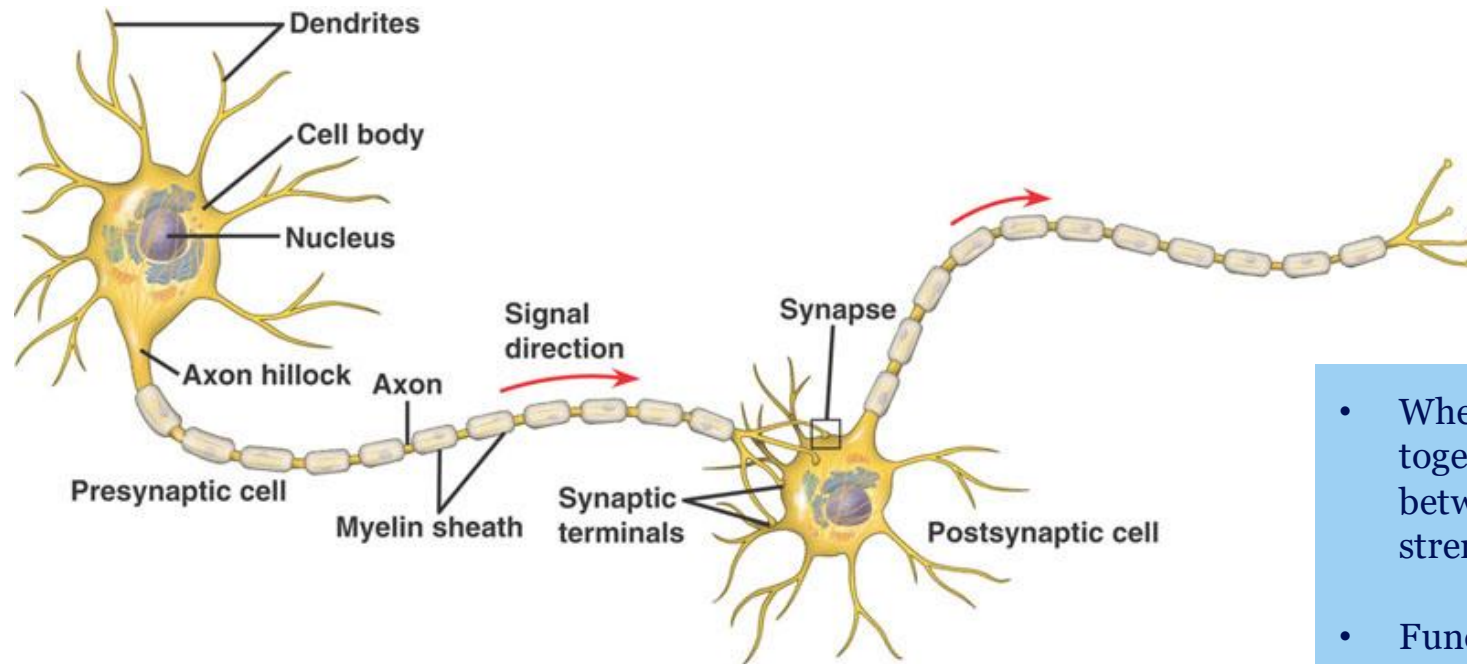
Donald Hebb wrote in 1949:

“When an axon in cell A is near enough to excite cell B and repeatedly and persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A’s efficiency in firing B is increased.”

Today this growth process is known as ***Hebbian learning***

Hebbian Learning

“When an axon in cell A is near enough to excite cell B and repeatedly and persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A’s efficiency in firing B is increased.”



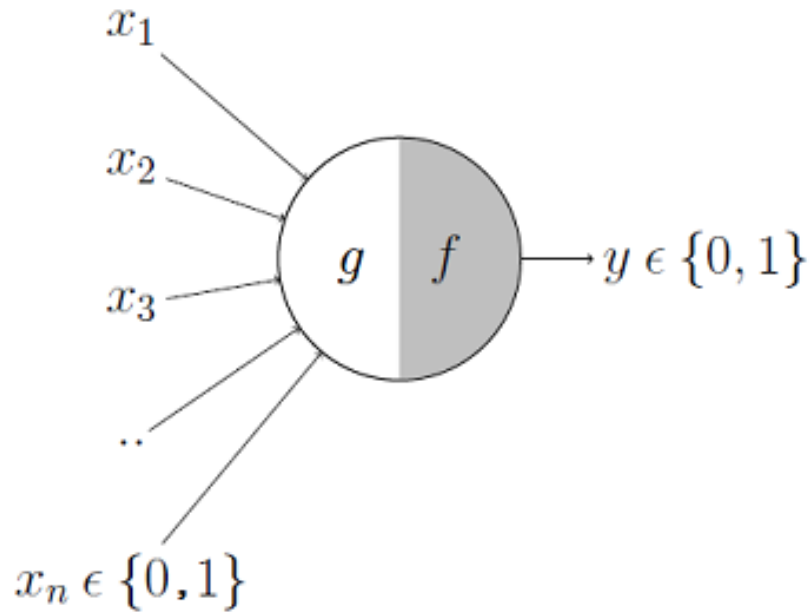
$$\Delta w_1(t) \propto x(t) y(t)$$

- When two neurons fire together, the connection between the neurons is strengthened.
- Fundamental operation for learning and memorization

Artificial (McCulloch-Pitts) Neuron

First McCulloch-Pitts model:

The first computational model of a neuron was proposed by Warren McCulloch (neuroscientist) and Walter Pitts (logician) in 1943.



2 PARTS:

- First part: $g \rightarrow$ takes an input and performs an aggregation
- Second part: $f \rightarrow$ decides based on the aggregated value

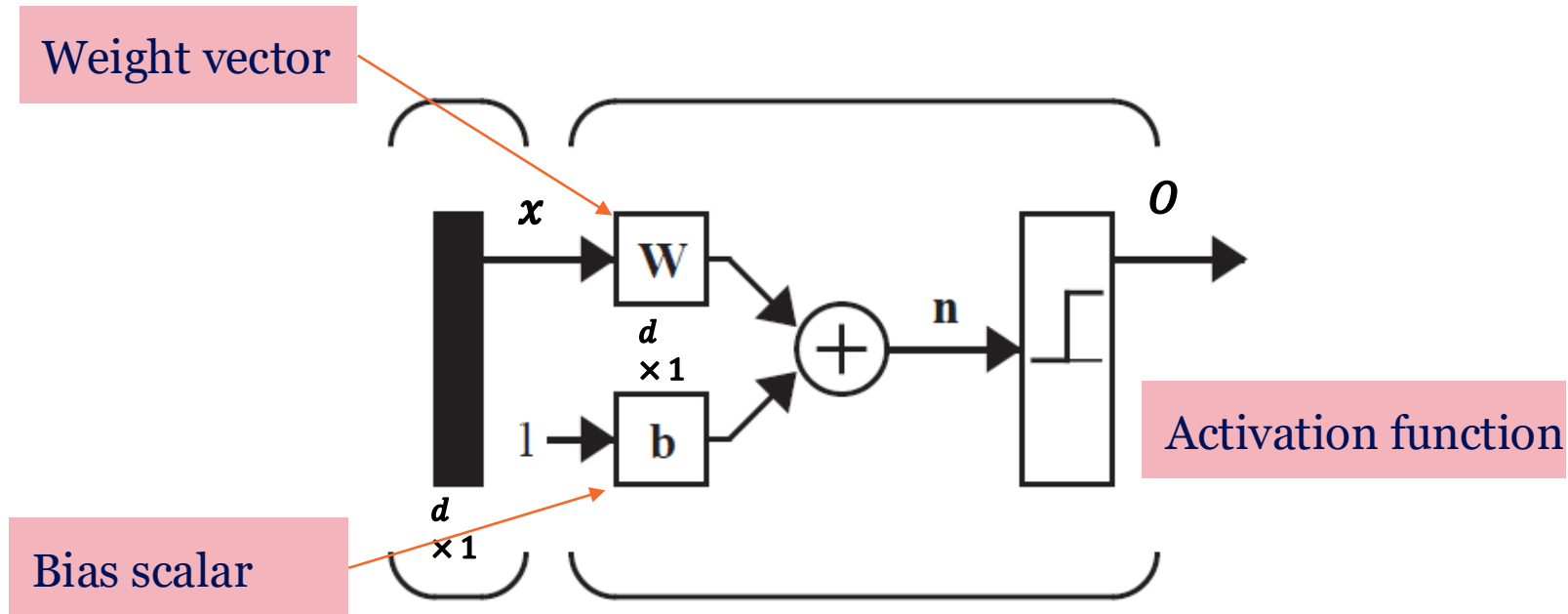
$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$

Artificial (McCulloch-Pitts) Neuron

Artificial neuron using a sign function (or Heaviside step function) as activation function; also called threshold unit

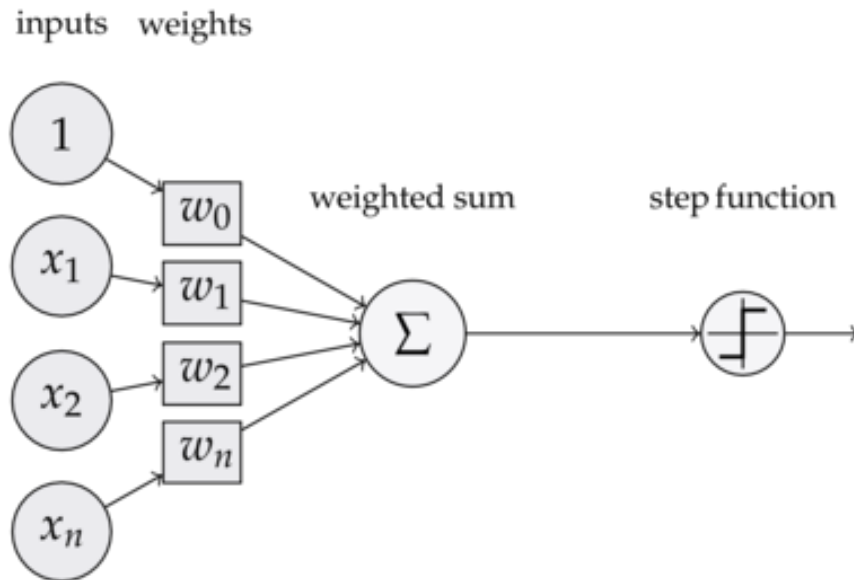
$$O = \text{sgn}(\vec{w}^T \vec{x} + b)$$



Artificial (McCulloch-Pitts) Neuron

Successive McCulloch-Pitts model (a.k.a. **simple perceptron**):

$$O = \text{sgn}(\vec{w}^T \vec{x} + w_0)$$
$$O = \text{sgn}\left(\sum_i w_i x_i\right), \quad \text{sgn}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$



WHAT IS NEW:

- Operates also on non-boolean values
- Each input connection is assigned a weight

Artificial Neuron

The McCulloch-Pitts model:

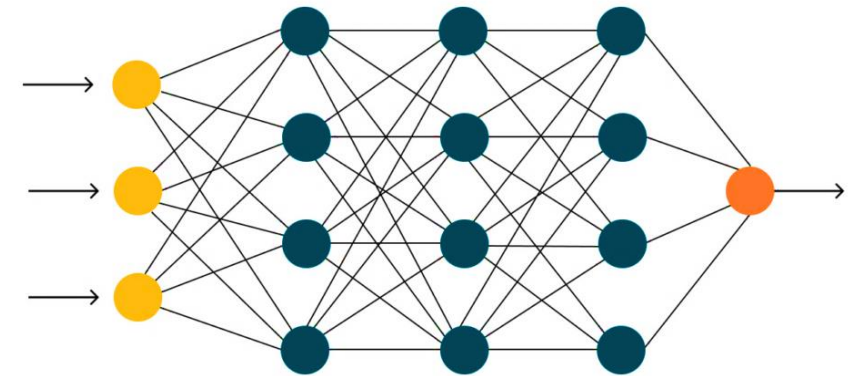
- **spikes** are interpreted as spike rates (firing rate);
- **synaptic strength** are translated as weights
- **excitation** means positive product between the incoming input and the corresponding synaptic weight
- **inhibition** means negative product between the incoming input and the corresponding synaptic weight

The perceptron algorithm is also called the single-layer perceptron, to distinguish it from a multilayer perceptron.

Artificial Neural Network

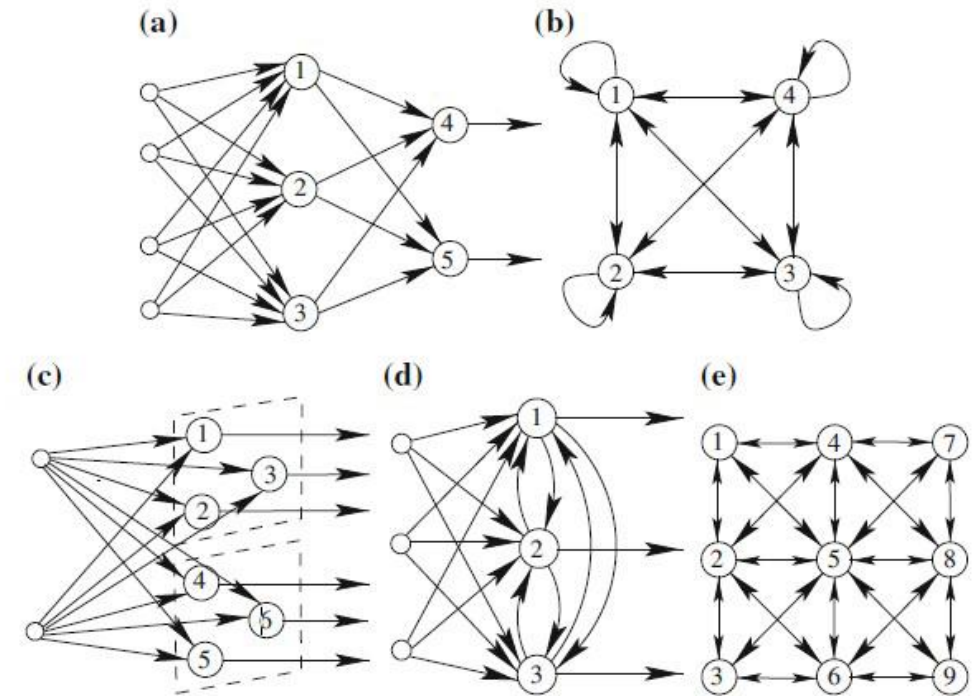
An Artificial Neural Network is a collection of connected artificial neurons and it is characterized by:

- **architecture/topology**: the nodes and the arcs connecting the neurons with corresponding weights;
- **node characteristics**: in particular, the activation function;
- **learning rules**: the algorithms used to train the Artificial Neural Network. in such a way that it produces the desired output for a given input.



Network Topologies

- The topology of the network defines the way in which neurons are connected and relate to each other.
- The architecture of an ANN plays a fundamental role in its functionality and performance.
- Every topology corresponds to different problems and different learning algorithms.



Network Topologies

A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

Backfed Input Cell

Input Cell

Noisy Input Cell

Hidden Cell

Probabilistic Hidden Cell

Spiking Hidden Cell

Output Cell

Match Input Output Cell

Recurrent Cell

Memory Cell

Different Memory Cell

Kernel

Convolution or Pool

Perceptron (P)



Feed Forward (FF)



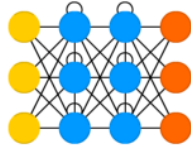
Radial Basis Network (RBF)



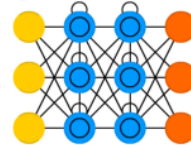
Deep Feed Forward (DFF)



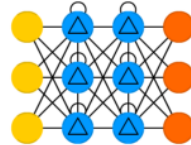
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)



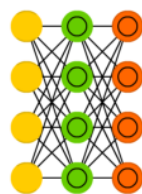
Gated Recurrent Unit (GRU)



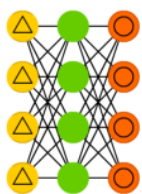
Auto Encoder (AE)



Variational AE (VAE)



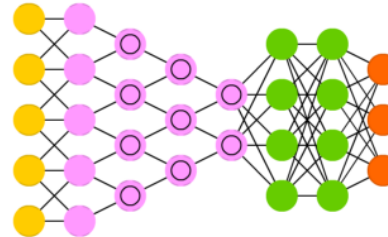
Denoising AE (DAE)



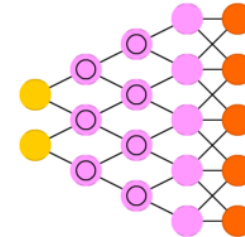
Sparse AE (SAE)



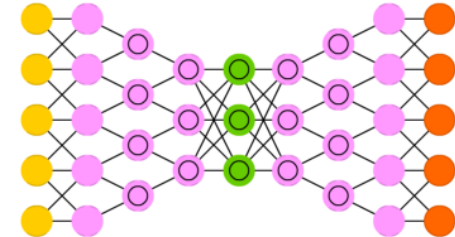
Deep Convolutional Network (DCN)



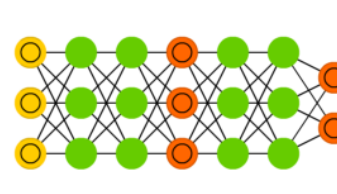
Deconvolutional Network (DN)



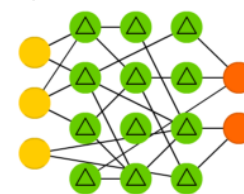
Deep Convolutional Inverse Graphics Network (DCIGN)



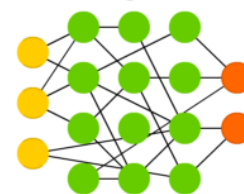
Generative Adversarial Network (GAN)



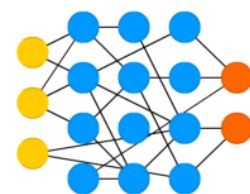
Liquid State Machine (LSM)



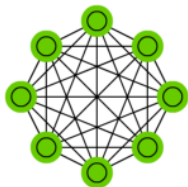
Extreme Learning Machine (ELM)



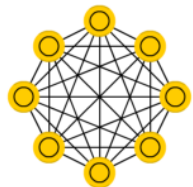
Echo State Network (ESN)



Markov Chain (MC)



Hopfield Network (HN)



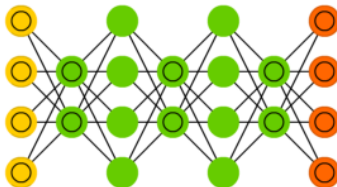
Boltzmann Machine (BM)



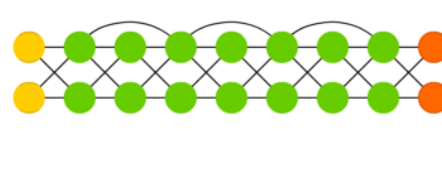
Restricted BM (RBM)



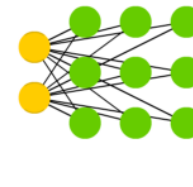
Deep Belief Network (DBN)



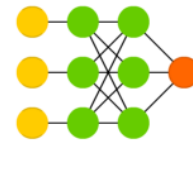
Deep Residual Network (DRN)



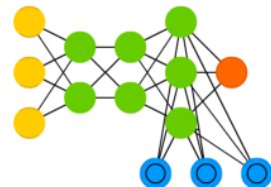
Kohonen Network (KN)



Support Vector Machine (SVM)

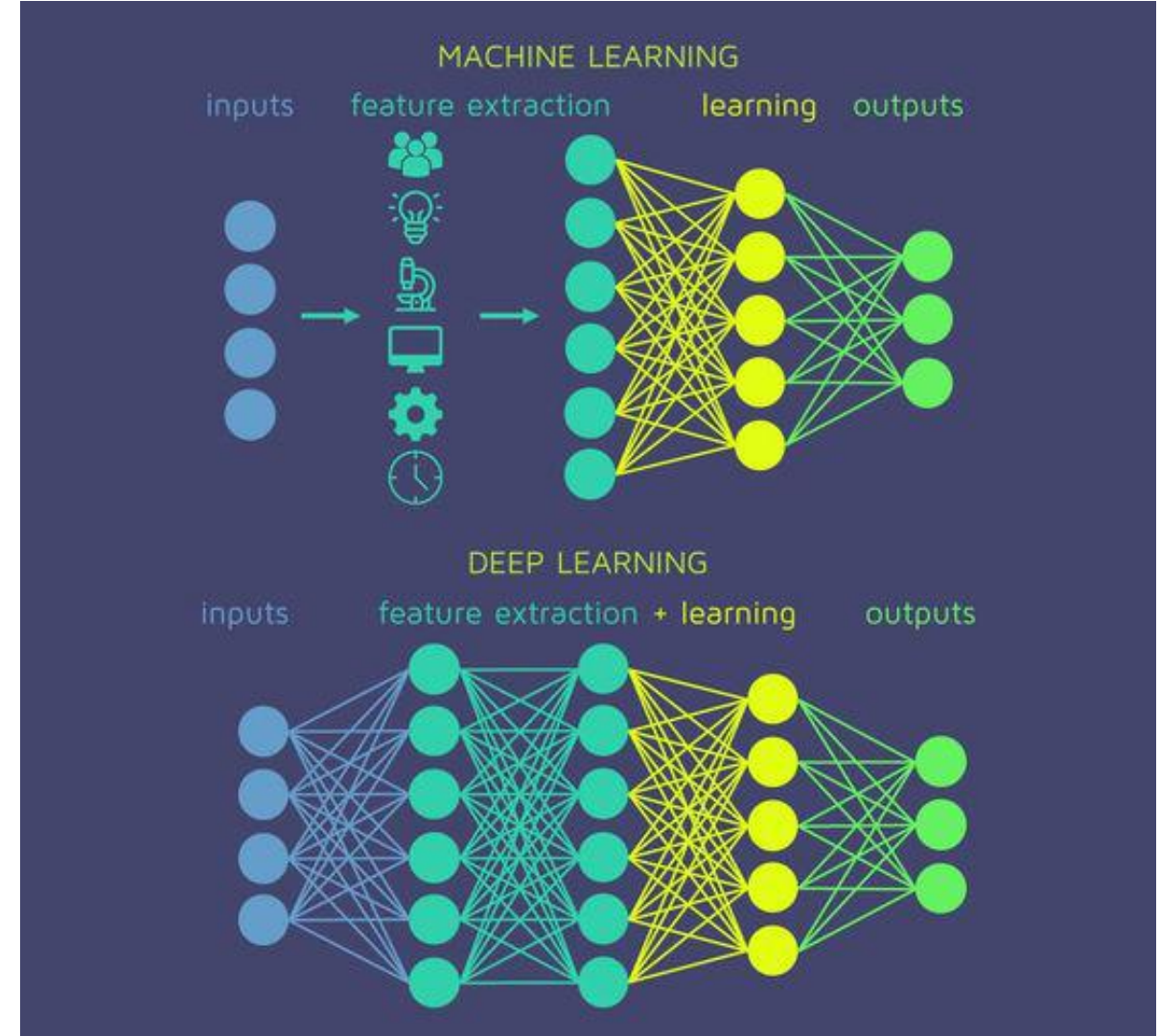


Neural Turing Machine (NTM)



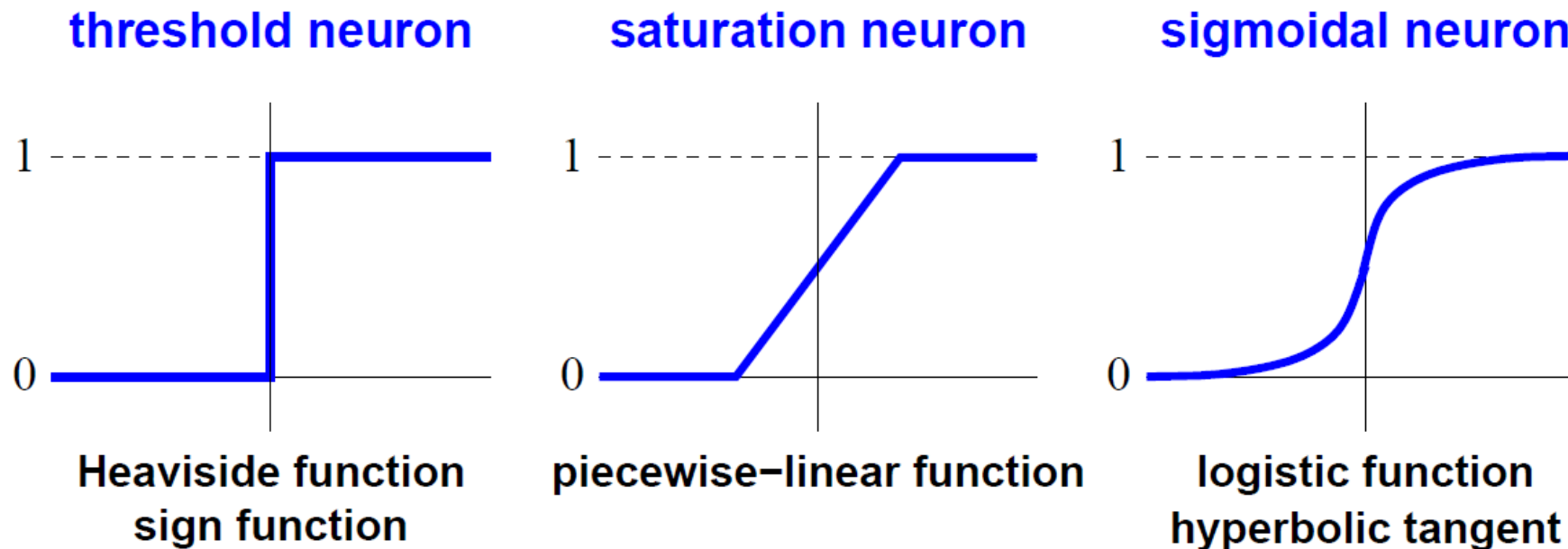
Network Topologies

- From a topological point of view, **Deep Neural Networks** are simply feedforward networks with many hidden layers
- However, the important difference is in the way the two models operate, i.e., how they **function** and **process information**.
- Deep networks **learn hierarchical representations** from data, meaning they **automatically extract complex features** at different layers.



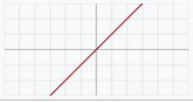
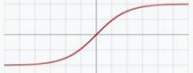
Activation Functions

- Their **aim** is to **transform the weighted sum of the input values in the final output** produced by the neurons.
- One can replace the threshold unit to other functions (non-linear): **Binary output $\rightarrow$ real-valued output**
- Common activation functions:



Activation Functions

But there are many more!

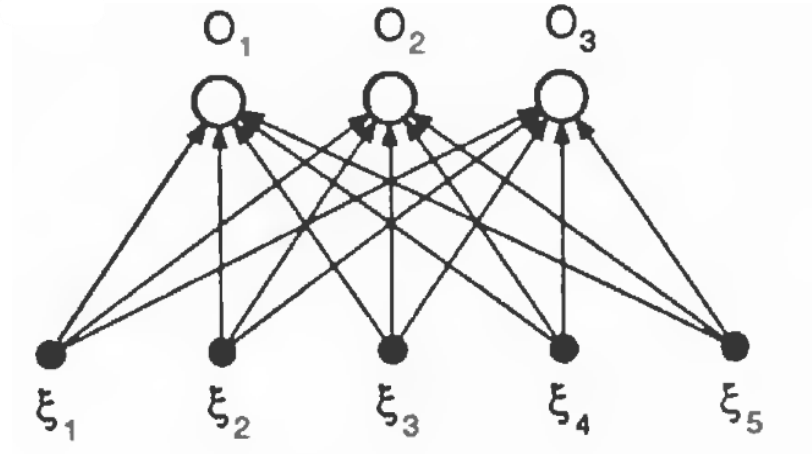
Name	Plot	Equation	Derivative (with respect to x)	Range	Order of continuity
Identity		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$	C^∞
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$	$\{0, 1\}$	C^{-1}
Logistic (a.k.a. Sigmoid or Soft step)		$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$ ^[1]	$f'(x) = f(x)(1 - f(x))$	$(0, 1)$	C^∞
TanH		$f(x) = \tanh(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$	$f'(x) = 1 - f(x)^2$	$(-1, 1)$	C^∞
Rectified linear unit (ReLU) ^[12]		$f(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases} = \max\{0, x\} = x \mathbf{1}_{x>0}$	$f'(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ 1 & \text{for } x > 0 \end{cases}$	$[0, \infty)$	C^0
Gaussian Error Linear Unit (GELU) ^[7]		$f(x) = x\Phi(x) = x(1 + \operatorname{erf}(x/\sqrt{2}))/2$	$f'(x) = \Phi(x) + x\phi(x)$	$(\approx -0.17, \infty)$	C^∞
SoftPlus ^[13]		$f(x) = \ln(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$	$(0, \infty)$	C^∞
Exponential linear unit (ELU) ^[14]		$f(\alpha, x) = \begin{cases} \alpha(e^x - 1) & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} f(\alpha, x) + \alpha & \text{for } x \leq 0 \\ 1 & \text{for } x > 0 \end{cases}$	$(-\alpha, \infty)$	$\begin{cases} C^1 & \text{when } \alpha = 1 \\ C^0 & \text{otherwise} \end{cases}$

Learning Rule

- ANNs can **generalize** = having the capability to familiarize with problems by means of **training** and later to **solve unknown** problems of the same class.
- **Training** is typically accomplished via examples. Then, the network parameters are adapted using a learning algorithm.
- **Learning** = ability to modify proper characteristics (e.g., changes of the weights and biases) according to changes in the environment.
- Artificial Neural Networks are trained by **epoch** = a complete run when the entire training set is presented and utilized by the learning algorithm.

Learning Rule: Simple Perceptron

- The simple perceptron has only input and output layers (no hidden layers).



- For each output node O_i , for $i = 1, \dots, M$, the output value is computed as

$$O_i = \text{sgn}(\vec{w}^i \vec{\xi} + b_i)$$

- Note that the threshold values b_i can be incorporated in the scalar product by introducing an additional input node with value always fixed to 1.

Learning Rule of the Simple Perceptron

Hebbian learning algorithm for perceptron

As the **computed output values are all independent** from each other, let's consider the case where we have $M = 1$ output.

Given a data set $(\vec{x}^1, y^1), (\vec{x}^2, y^2), \dots, (\vec{x}^P, y^P)$, $\vec{x}^p \in \mathbb{R}^d, y^p \in \{-1, 1\}$, we denote Perceptron's output by

$$O^p = \text{sgn}(\vec{w}^T \vec{x}^p + b), \quad p = 1, \dots, P.$$

The aim is to identify values of w^i and b in such a way that:

$$O^p = y^p \quad \text{for } p = 1, \dots, P$$

($p = \#$ target patterns)

Learning Rule of the Simple Perceptron

- Training set: $(\vec{x}^1, y^1), (\vec{x}^2, y^2), \dots, (\vec{x}^P, y^P), \vec{x}^p \in \mathbb{R}^d, y^p \in \{-1, 1\}$,
- Corresponding output on each input pattern:

$$O^p = \text{sgn}(\vec{w}^T \vec{x}^p + b), \quad p = 1, \dots, P.$$

- **Learning Rule:** Perform the calculation on the whole training set until the weights converge:

$$\vec{w}' = \vec{w} + \Delta \vec{w}$$

$$\Delta w_j = \begin{cases} 2\eta y_p x_j^p & \text{if } O_p \neq y_p \\ 0 & \text{otherwise} \end{cases}$$

Don't worry, we will explore this in more detail later.

where $0 < \eta < 1$ is the **learning rate**, a small constant (e.g., 0.01), and j = index for input nodes.

Learning Rule of the Simple Perceptron

The aim is to identify values of w^i and b in such a way that:

$$\text{sgn}(\vec{w}^T \vec{x}^p) = O^p = y^p \quad \text{for } p = 1, \dots, P$$

(p = # target patterns)

This means that the weight vector must be chosen so that the projection of the pattern $\vec{x}^p$ onto it has the **same sign** as y^p .

The boundary between positive and negative projections onto $\vec{w}$ is the hyperplane: $\vec{w}^T \vec{x}^p = 0$, which passes through the origin and is perpendicular to $\vec{w}$.

Hence, for correct operation, this plane should divide the inputs that have positive and negative targets.

Can we solve a binary classification task?

Yes!

Learning Rule of the Simple Perceptron

The aim is to identify values of w^i and b in such a way that:

$$\text{sgn}(\vec{w}^T \vec{x}^p) = O^p = y^p \quad \text{for } p = 1, \dots, P$$

(p = # target patterns)

This means that the weight vector must be chosen so that the projection of the pattern $\vec{x}^p$ onto it has the **same sign** as y^p .

The boundary between positive and negative projections onto $\vec{w}$ is the hyperplane: $\vec{w}^T \vec{x}^p = 0$, which passes through the origin and is perpendicular to $\vec{w}$.

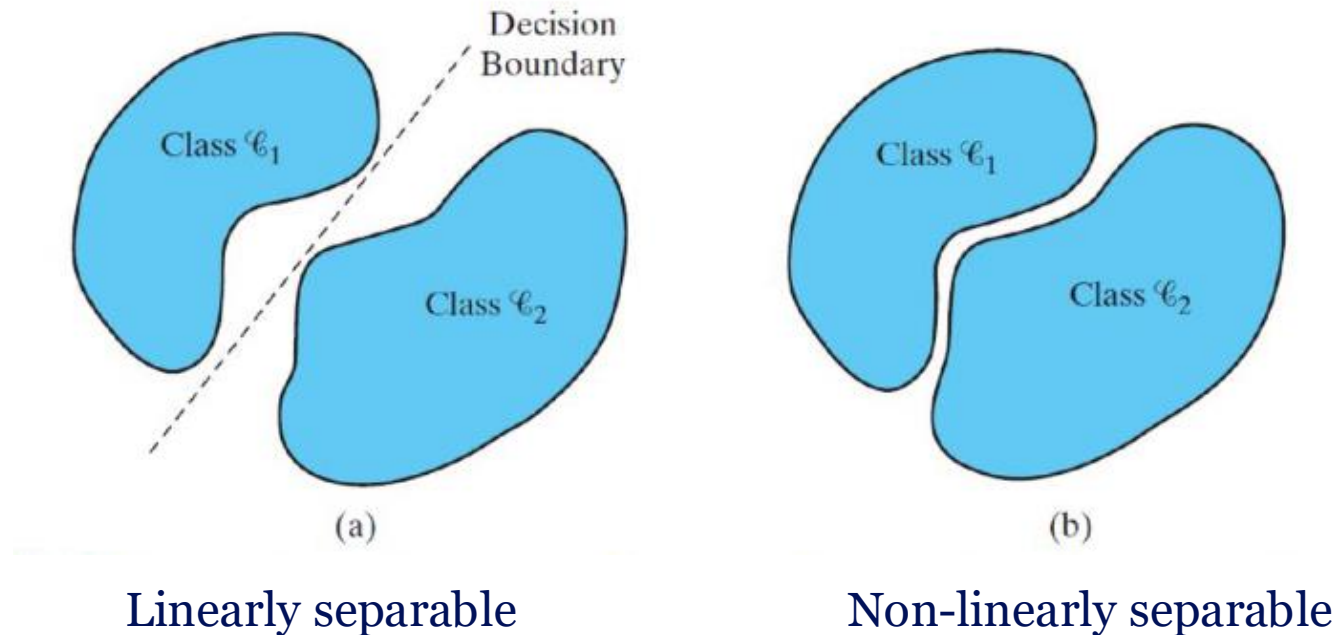
Hence, for correct operation, this plane should divide the inputs that have positive and negative targets.

Can we solve a binary classification task?

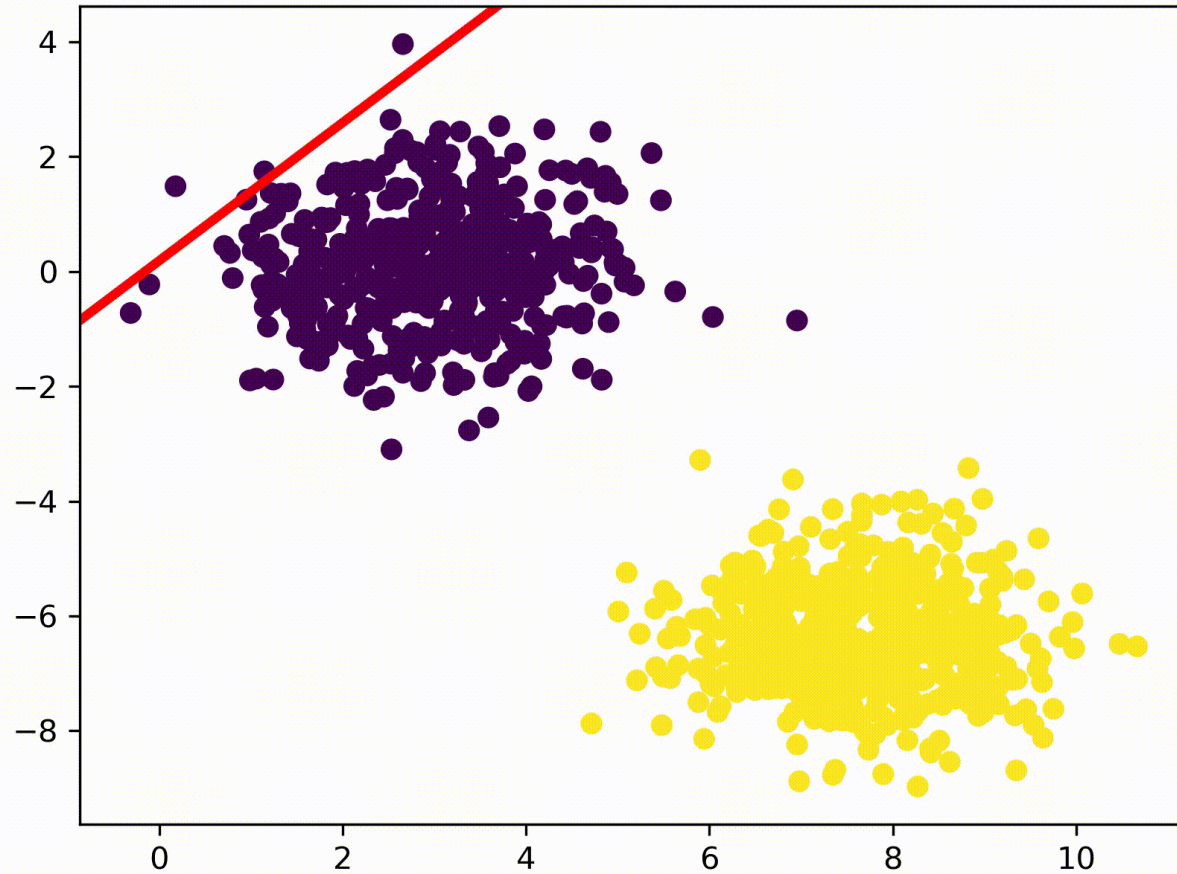
Yes!

Binary Classification

Given a data set, want to “draw” a **curve** to separate two classes of samples:



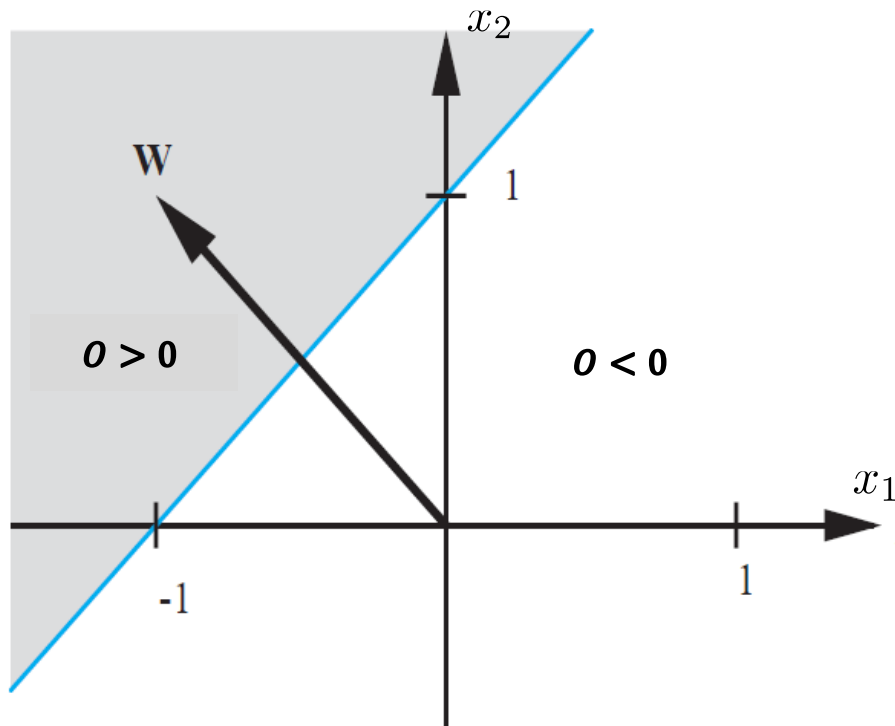
Example of linearly separable sets



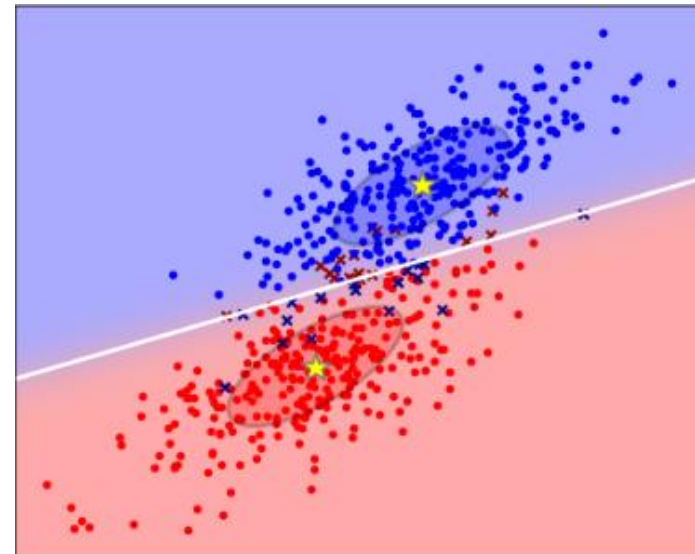
Binary Classification

The simple perceptron can only solve a ***linearly separable*** task.

Why? Recall the decision boundary of a perceptron.



No cases in between...



Linear Separability

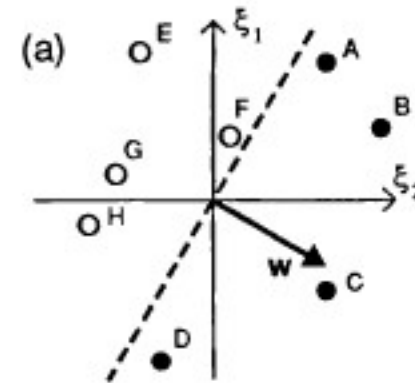
However, it is often convenient to use an alternative representation. Let

$$\hat{x}^p = y_p x^p, \quad p = 1, \dots, P.$$

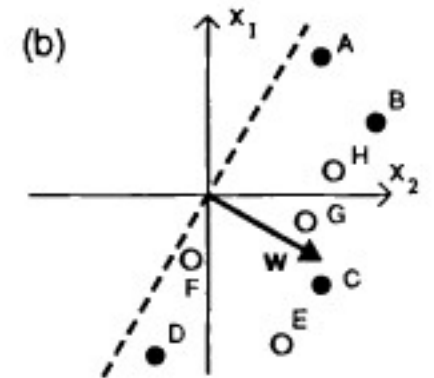
The classification problem requires now to determine a vector w such that

$$w^T \hat{x}^p > 0, \quad p = 1, \dots, P.$$

Here a geometrical illustration of the two cases:

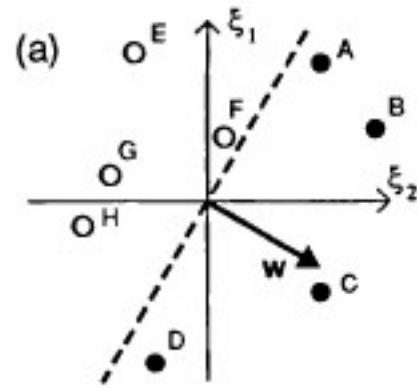


$$y_p = \text{sgn}(w^T x^p)$$

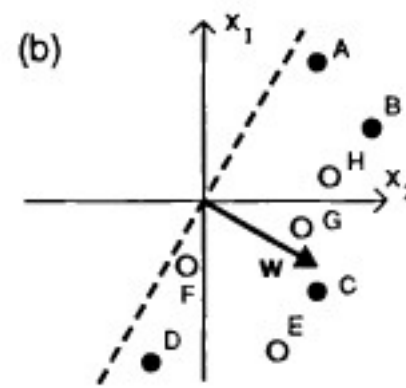


$$w^T \hat{x}^p > 0$$

Linear Separability



$$y_p = \text{sgn}(w^T x^p)$$



$$w^T \hat{x}^p > 0$$

These two conditions can be satisfied only if data are linearly separable, i.e., there exist a weight vector such that

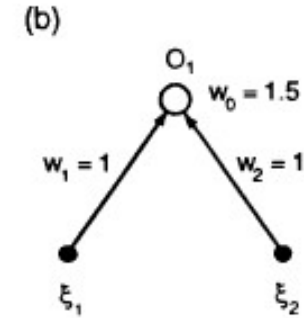
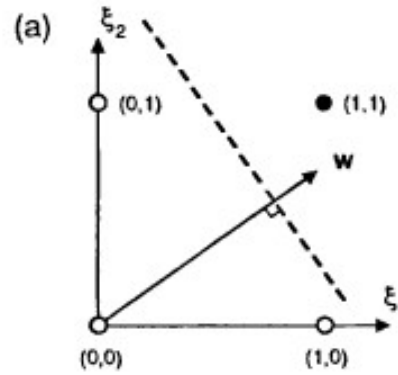
$$\begin{aligned} w^T x &> 0 & \text{if } y &= 1 \\ w^T x &< 0 & \text{if } y &= -1. \end{aligned}$$

Let's consider the AND logical function as an example.

Linear Separability: AND

The AND table is:

x_1	x_2	y
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1



Here, data are linearly separable, and a solution is given by

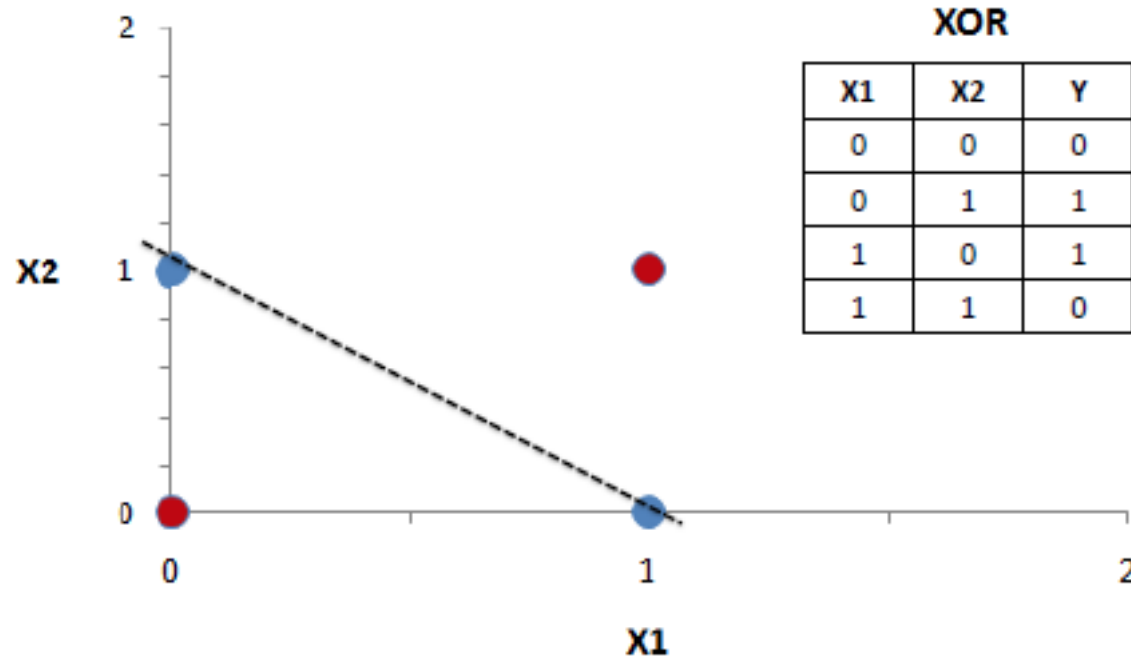
$$w_1 = 1, w_2 = 1, w_3 = \theta = 1.5$$

In fact

x_1	x_2	y	$w_1x_1 + w_2x_2 - w_3$
-1	-1	-1	-3.5
-1	1	-1	-1.5
1	-1	-1	-1.5
1	1	1	0.5

How about non-linearly separable data?

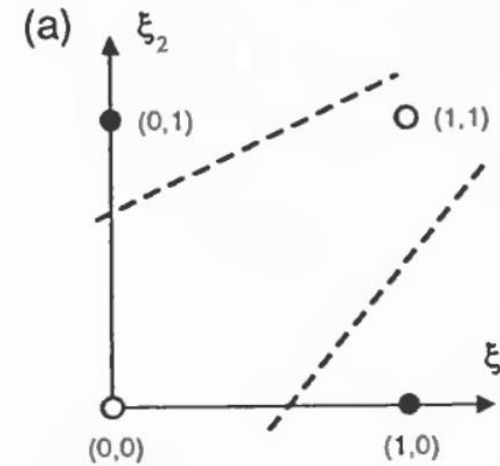
The eXclusive OR problem (XOR)



Non-linearly separable data: XOR

The XOR table is:

x_1	x_2	y
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1



The XOR function is not-linearly separable, in fact:

x_1	x_2	y	$w_1x_1 + w_2x_2 - w_3$	
-1	-1	-1	$-w_1 - w_2 - w_3 < 0$	(a)
-1	1	1	$-w_1 + w_2 - w_3 > 0$	(b)
1	-1	1	$w_1 - w_2 - w_3 > 0$	(c)
1	1	-1	$w_1 + w_2 - w_3 < 0$	(d)

Non-linearly separable data: XOR

The XOR function is not-linearly separable, in fact:

x_1	x_2	y	$w_1x_1 + w_2x_2 - w_3$	
-1	-1	-1	$-w_1 - w_2 - w_3 < 0$	(a)
-1	1	1	$-w_1 + w_2 - w_3 > 0$	(b)
1	-1	1	$w_1 - w_2 - w_3 > 0$	(c)
1	1	-1	$w_1 + w_2 - w_3 < 0$	(d)

From (d) and (b) it follows

$$w_1 + w_2 < w_3 < -w_1 + w_2$$

Hence, $w_1 < 0$.

From (a) and (c) it follows

$$-w_1 - w_2 < w_3 < w_1 - w_2$$

From which $w_1 > 0$. **→ CONTRADICTION!**

How about non-linearly separable data?

The eXclusive OR problem (XOR)

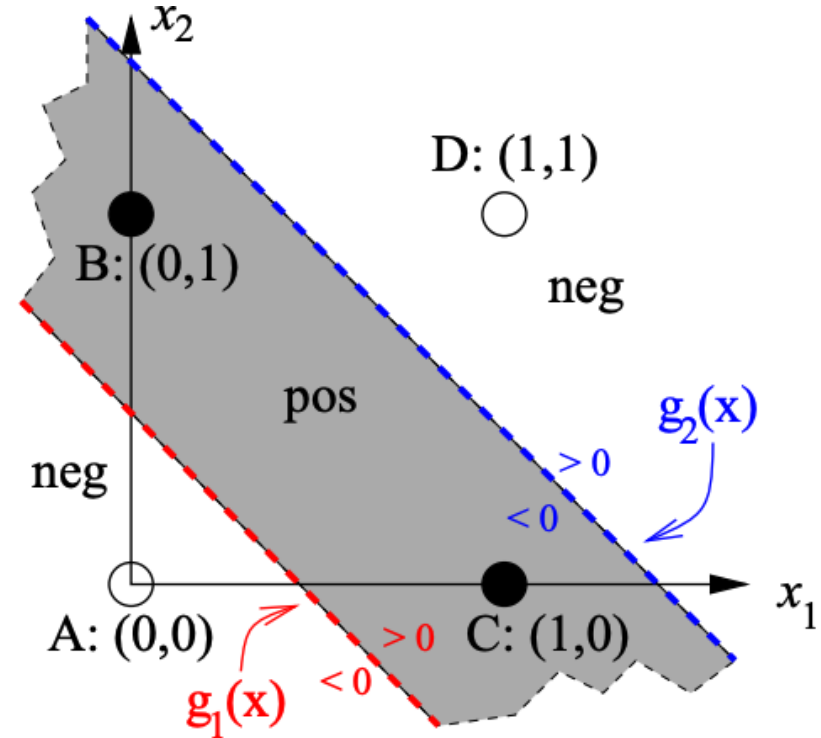
- Cannot be solved with one decision boundary/function
- What about two?

$$g_1(\vec{x}) = 1 \cdot x_1 + 1 \cdot x_2 - 1/2$$

$$g_2(\vec{x}) = 1 \cdot x_1 + 1 \cdot x_2 - 3/2$$

Positive = $g_1(\vec{x}) > 0$ AND $g_2(\vec{x}) < 0$

Negative = $g_1(\vec{x})g_2(\vec{x}) > 0$



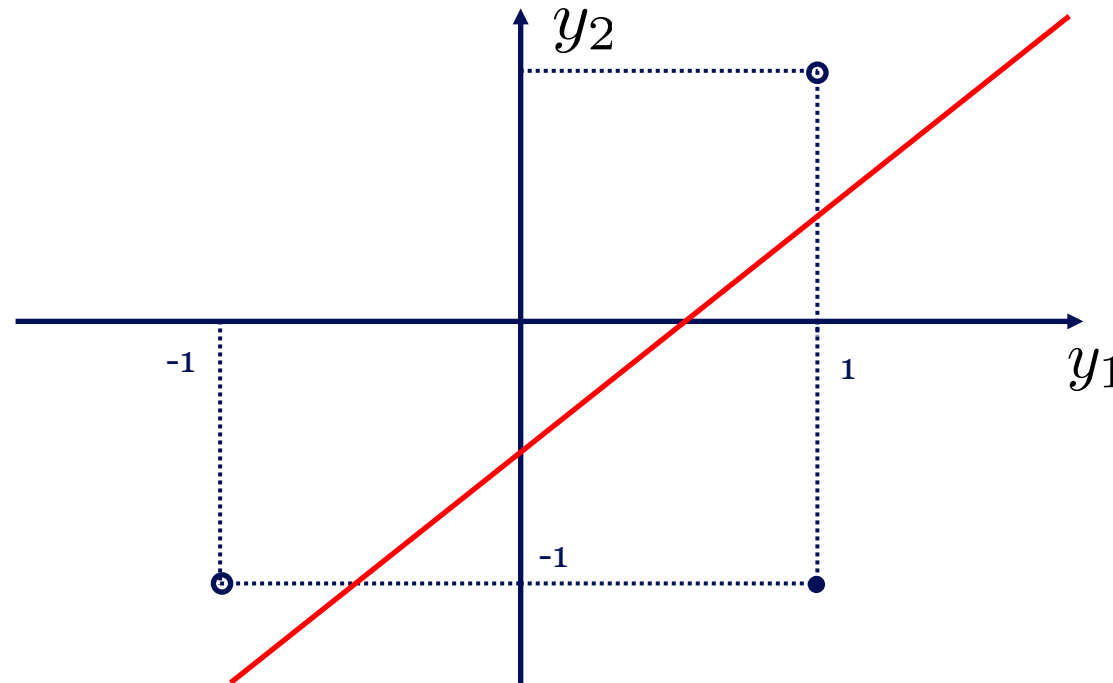
How about non-linearly separable data?

The eXclusive OR problem (XOR)

- What if we pass g_1, g_2 into a threshold function?

$$y_1 = \text{sgn}(g_1(\vec{x})), y_2 = \text{sgn}(g_2(\vec{x}))$$

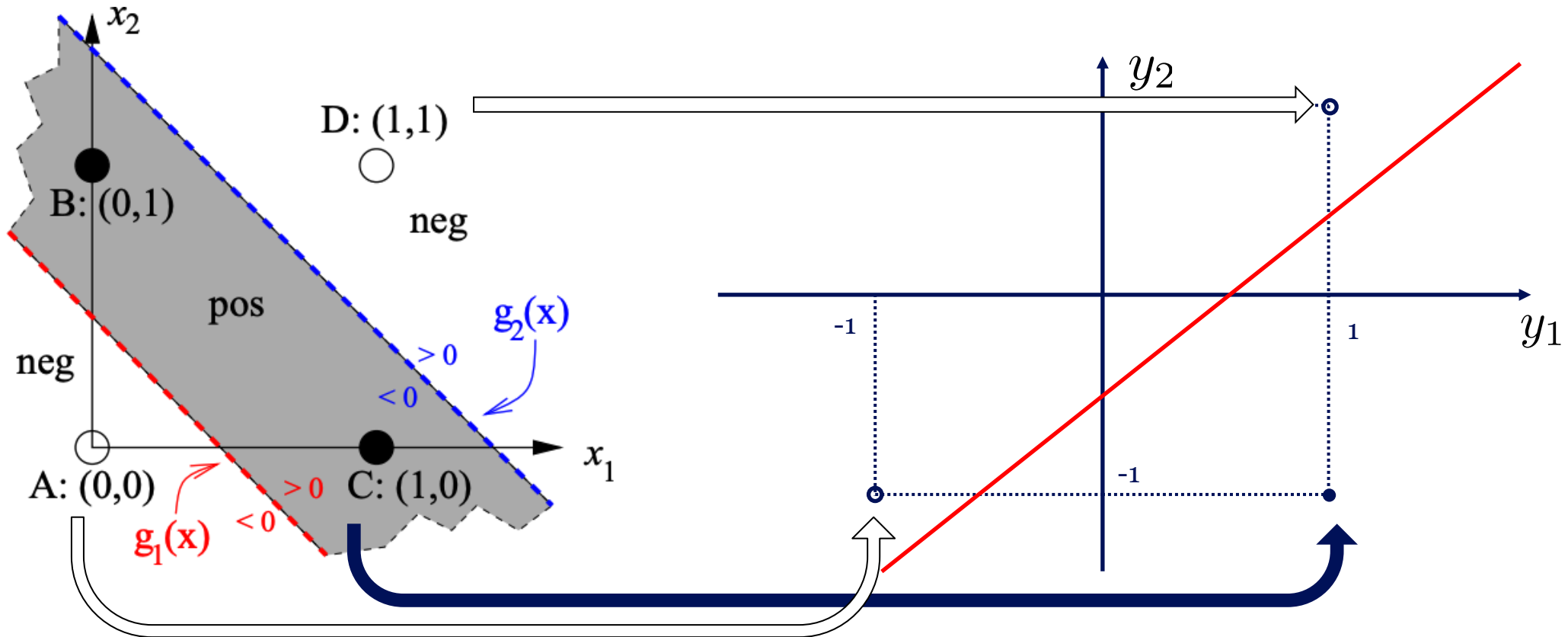
- Linearly separable!



How about non-linearly separable data?

The eXclusive OR problem (XOR)

- We map all vectors $\vec{x}$ to $\vec{y}$ such that the classes are linearly separable in the new space



Learning algorithm for the Perceptron

- Let's consider a dataset with P patterns and train a simple perceptron with 1 output node that can take binary values $y_p = \{-1, 1\}$.
- Let w_j be the current weights on the arcs connecting the j -th input node with the only input node ($j = 0, \dots, N$).
- Suppose that one training point is presented to the network, and the output is computed: $O_p = \text{sgn}(w^T x^p)$

Learning algorithm for the Perceptron

- The update formula is:

$$w_j' = w_j + \Delta_j$$

where

$$\Delta_j = \begin{cases} 2\eta y_p x_j^p & \text{if } O_p \neq y_p \\ 0 & \text{otherwise} \end{cases}$$

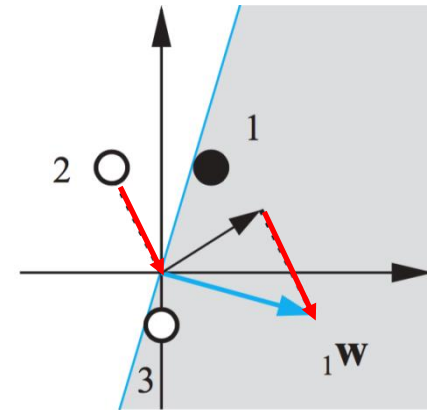
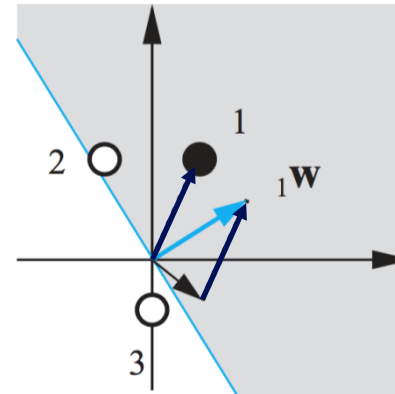
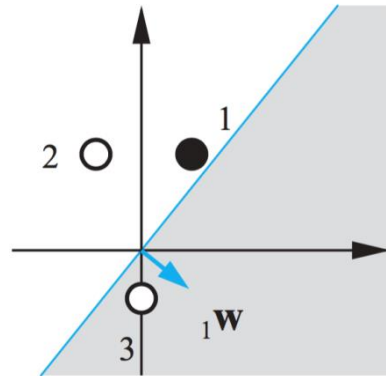
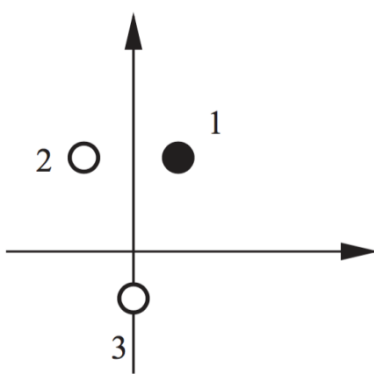
- Other possible choices for Δ_j are (all equivalent – **verify that**):

$$\Delta_j = \eta(1 - O_p y_p) x_j^p y_p \quad \text{and} \quad \Delta_j = \eta(y_p - O_p) x_j^p$$

An example

On a 2-D space, $\eta = \frac{1}{2}$, $b = 0$. Black=1, white=-1

$$\Delta_j = \begin{cases} 2\eta y_p x_j^p & \text{if } O_p \neq y_p \\ 0 & \text{otherwise} \end{cases}$$



What's the next step?

Perceptron: optimality

A better alternative to just asking that the sign of the predicted and the expected outputs is the same, it is sometimes convenient their product to be larger than a margin:

$$y_p w^T x^p > N\rho$$

for some fixed positive scalar ρ .

Not that the the number of input nodes N is included in the right term to keep the margin size fixed for any number of input units.

Perceptron: optimality

Hence, we can change the update rule into:

$$\Delta_j = \eta \Theta(N\rho - y_p w^T x^p) x_j^p y_p \quad j = 1, \dots, N + 1$$

where $\Theta(y) = \begin{cases} 1 & \text{if } y \geq 0 \\ 0 & \text{otherwise} \end{cases}$ is the unit step function.

Previous update formula:

$$\Delta_j = \eta(1 - O_p y_p) x_j^p y_p$$

Perceptron: optimality

Using again the quantities $\hat{x}^p = y_p x^p$, the condition

$$y_p w^T x^p > N\rho$$

can be rewritten as

$$w^T \hat{x}^p > N\rho$$

and the update formula $\Delta = \eta \Theta(N\rho - y_p w^T x^p) x^p y_p$ can be simplified in

$$\Delta = \eta \Theta(N\rho - w^T \hat{x}^p) \hat{x}^p$$

Hence, when the condition in **green** is not satisfied by a training point, the weights are modified by adding a multiple of the vector $\hat{x}^p$.

Perceptron: optimality

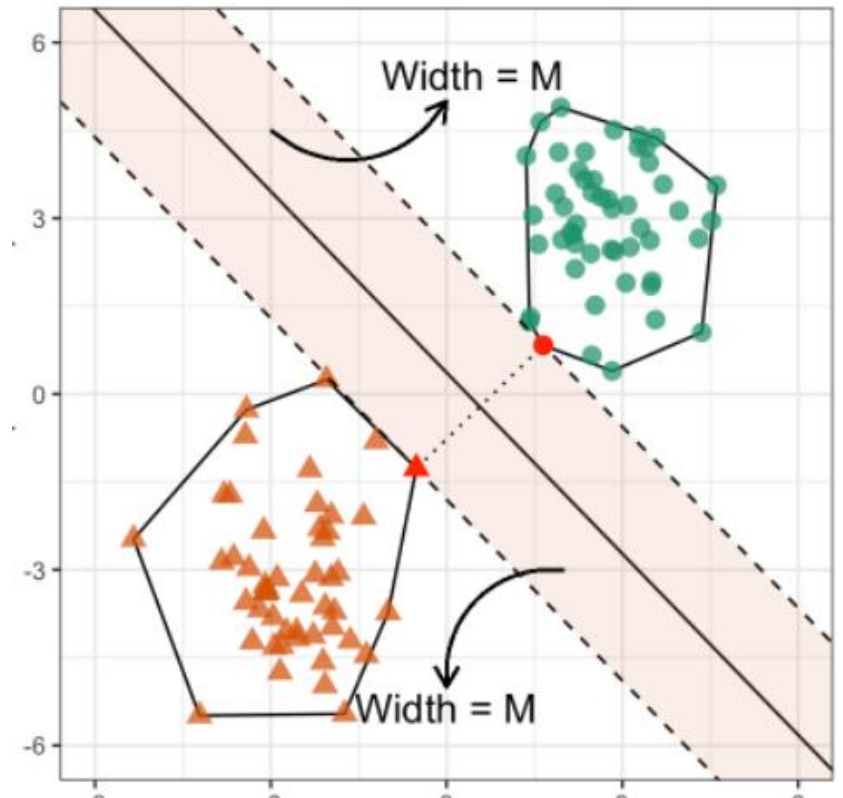
Optimality of Hebbian learning for linear binary classification

“Margin” of the decision boundary:

The closest point to the decision boundary among all correctly classified ones

Mathematically:

$$M(\vec{w}) = \frac{1}{\|\vec{w}\|} \min_{p \in [1..P]} \vec{w}^T \hat{x}^p$$

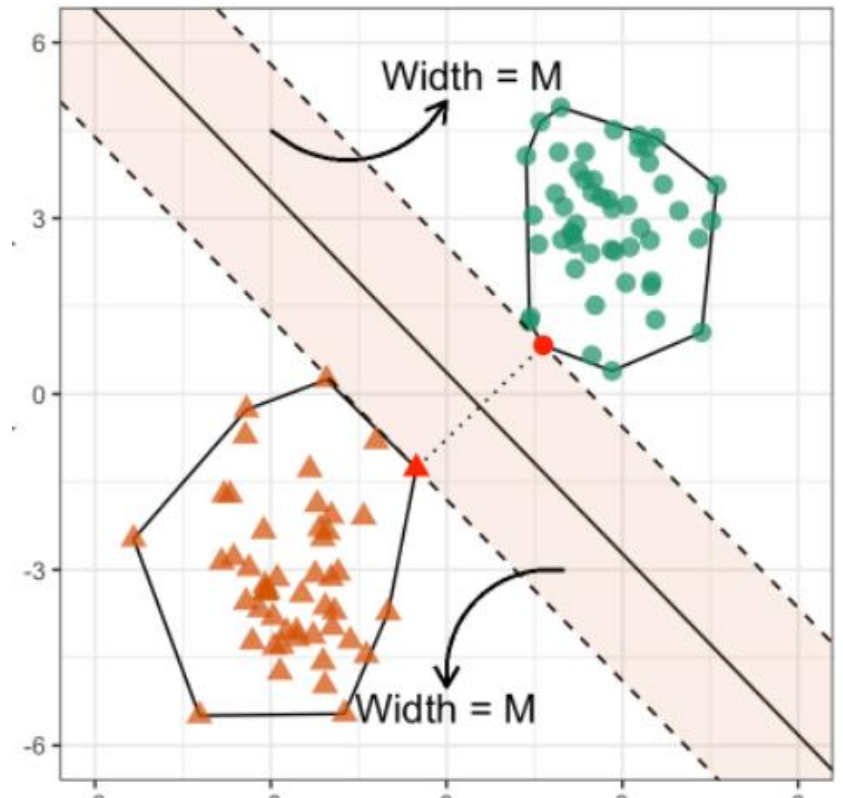


Perceptron: optimality

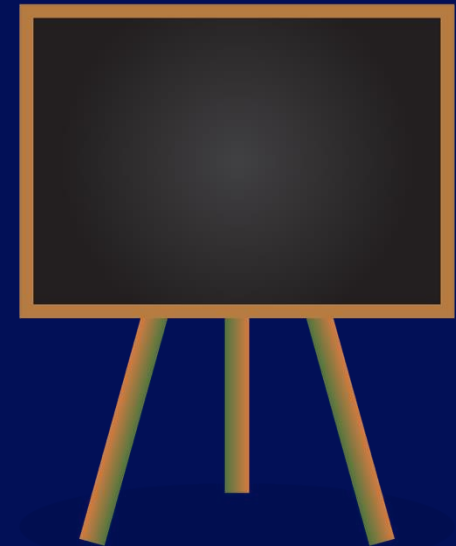
- The ***optimal weight vector*** is the one that maximizes the “margin”, i.e.,

$$\vec{w}^* = \arg \max_{\vec{w}} M(\vec{w}) = \arg \max_{\vec{w}} \min_{p \in [1..P]} \frac{\vec{w}^T \vec{x}^p}{\|\vec{w}\|}$$

- ***A maximin problem***
- The key idea in support vector machine



Proof of Convergence of the Learning Rule on the blackboard

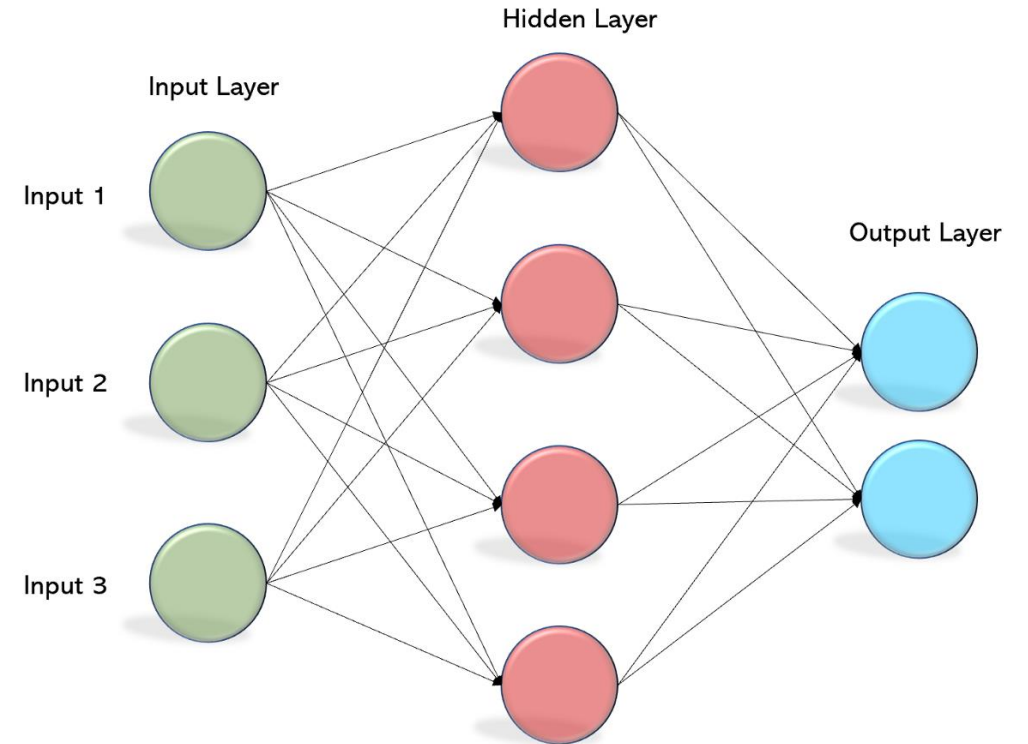


Multi-Layer Perceptron

How to implement this?

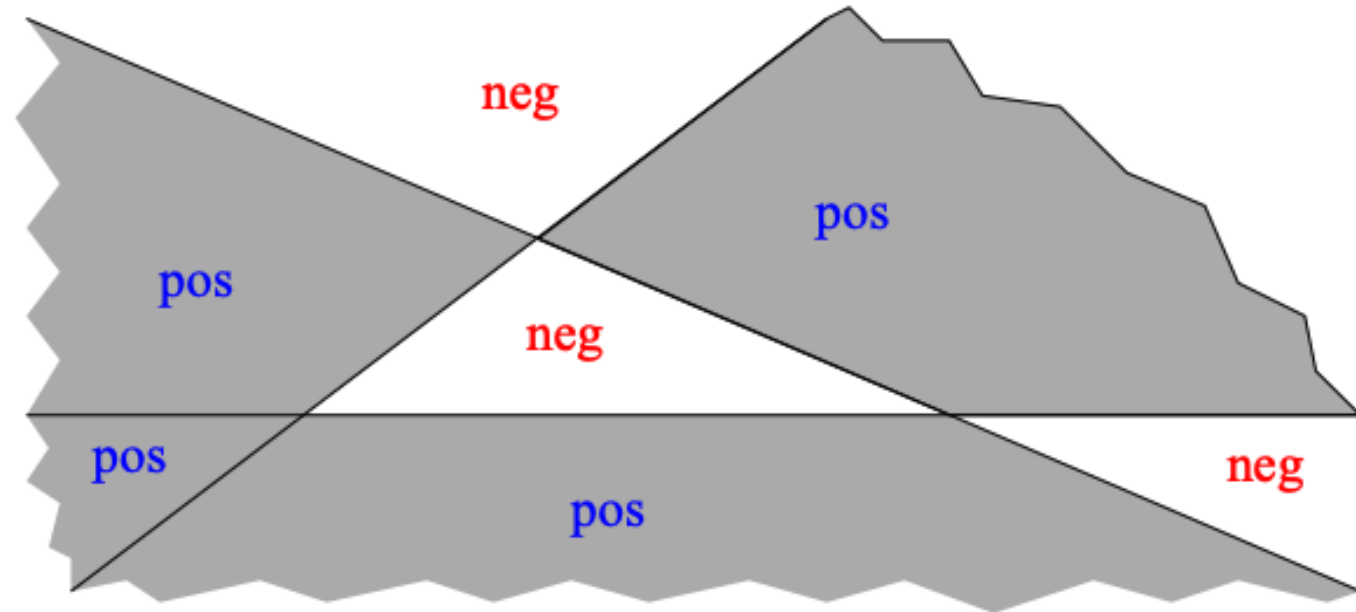
Multi-layer Perceptron

- The hidden layer (first layer) functions to ***extract features***
- The output layer (last layer) implements the decision function



Multi-Layer Perceptron

By adding up to 2 hidden layers of perceptrons, can represent any union of intersection of half-spaces.



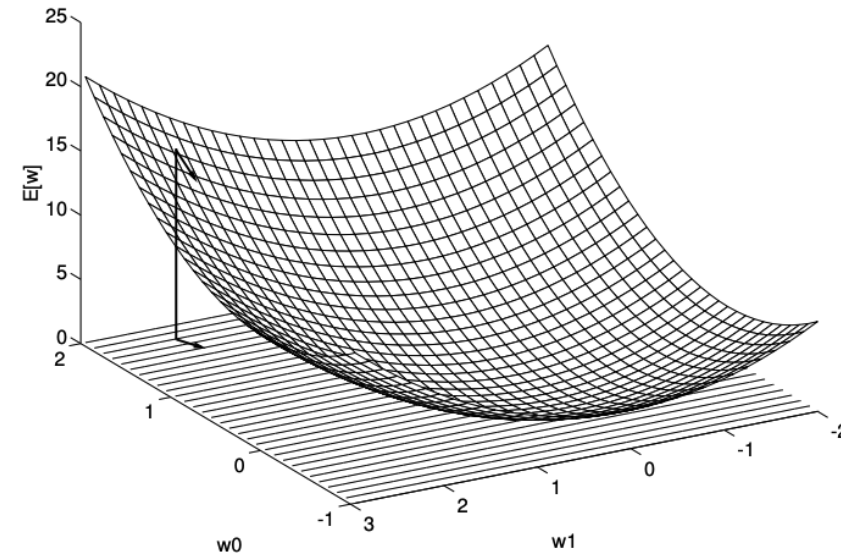
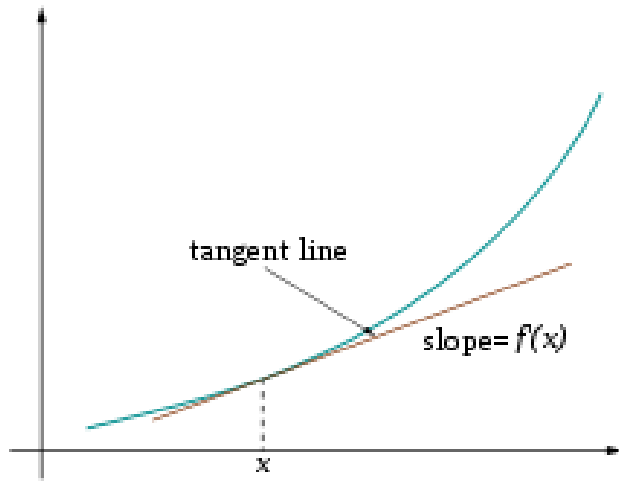
Multi-Layer Perceptron - how to train?

Perceptron revisited

Single perceptron: $O^p = \text{sgn}(\vec{w}^T \vec{x}^p + b), \quad p = 1, \dots, P$

Define a loss function: $L(\vec{w}, \vec{x}^p) = \frac{1}{2} (y_p - \vec{w}^T \vec{x}^p)^2$ (we here consider the case of linear units: $g(h) = h$)

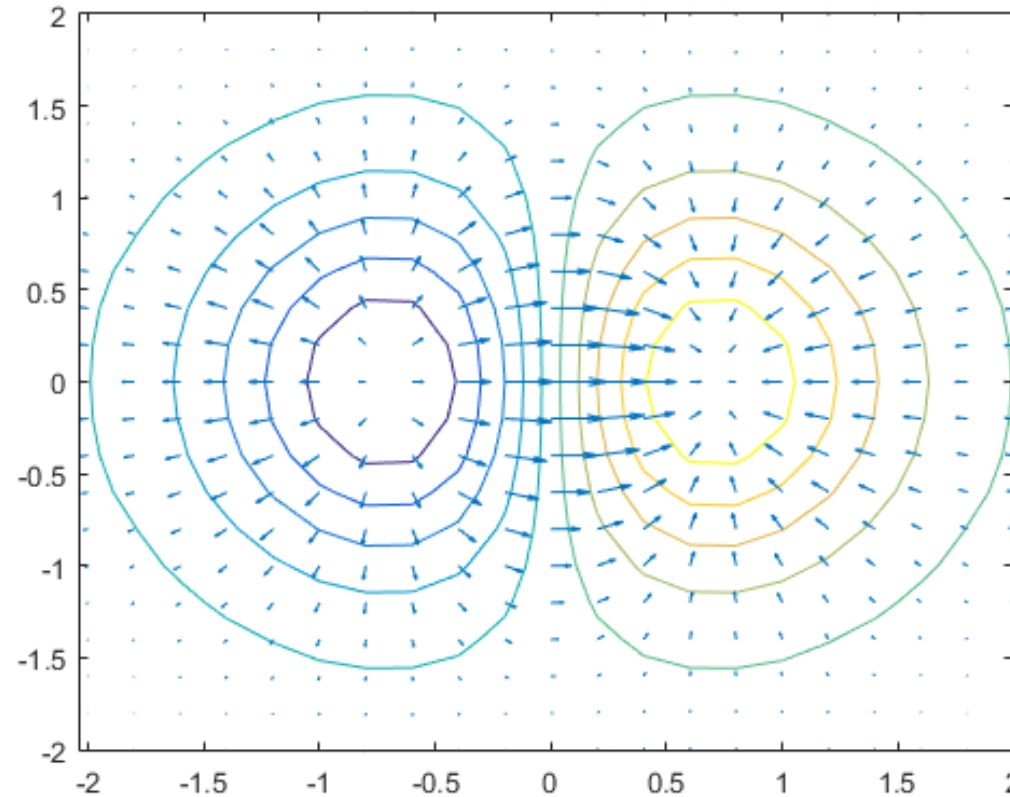
How does the loss change w.r.t. the weights? – gradients! $\rightarrow \frac{\partial L}{\partial w_k}$



Gradient field - Example

$$\nabla L = \frac{\partial L}{\partial \vec{w}} = \left(\frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_d} \right)^\top$$

$$f(x, y) = xe^{-x^2-y^2}$$



Multi-Layer Perceptron - how to train?

Perceptron revisited

Gradient is the steepest direction along which the loss increases

$$\begin{aligned}\frac{\partial L}{\partial w_k} &= \frac{\partial}{\partial w_k} \frac{\left(y_p - \sum_{j=1}^d w_j x_j^p\right)^2}{2} \\ &= -\left(y_p - \sum_{j=1}^d w_j x_j^p\right) x_k^p \\ &= -(y_p - o_p) x_k^p\end{aligned}$$

Does it look familiar? 😊

Multi-Layer Perceptron - how to train?

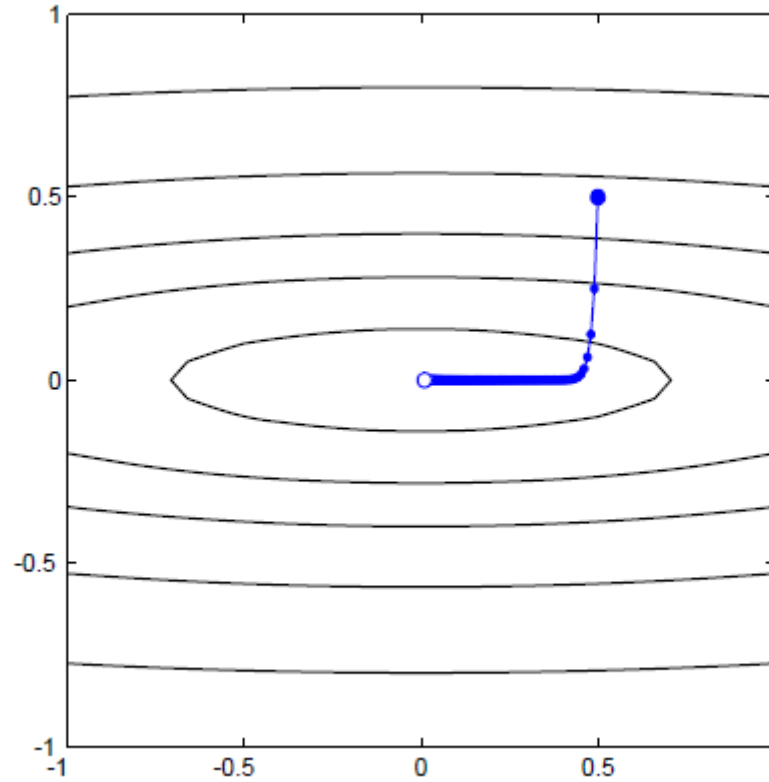
Perceptron revisited

Using the gradient to update the weights:

$$w_k \leftarrow w_k - \eta \frac{\partial L}{\partial w_k}$$
$$\Delta w_k = -\eta \frac{\partial L}{\partial w_k} = \eta (y_p - o_p) x_k^p$$

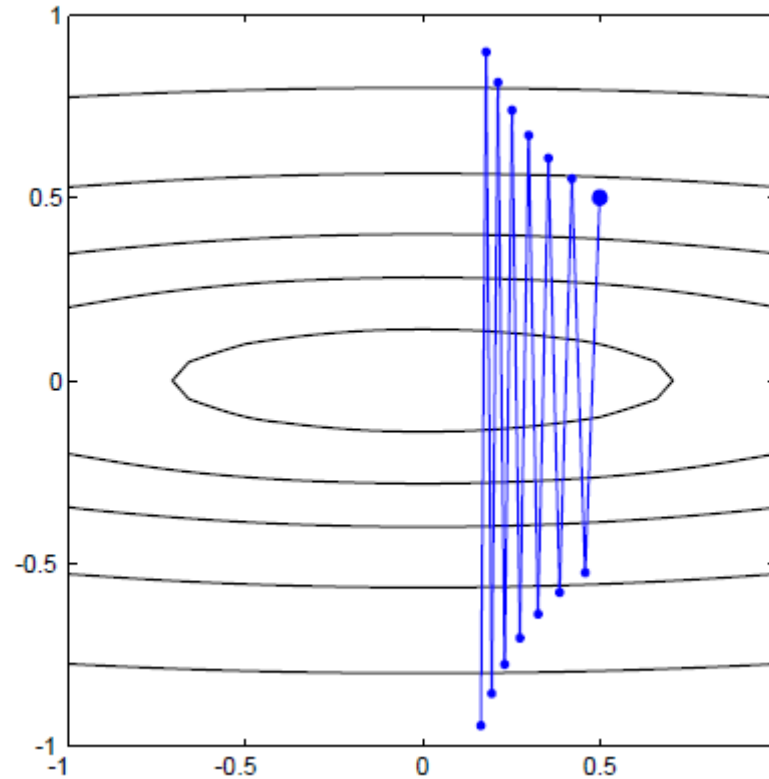
- We want to minimize the loss, hence subtracting the gradient from the weights – **Gradient Descent**
- The step-size η is necessary (next slides)
- We re-discover Hebb's rule!

Effect of the step-size: example



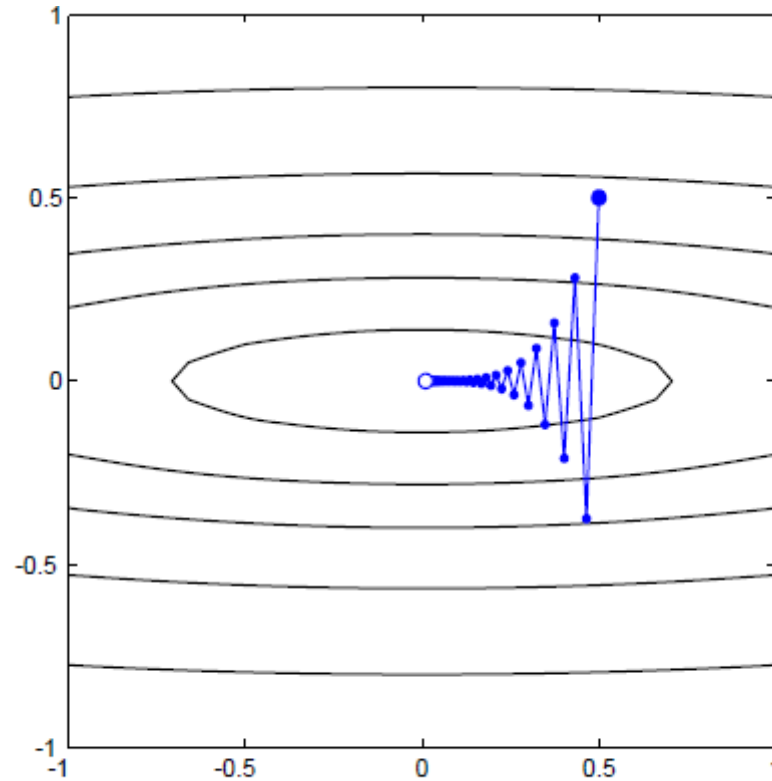
Trajectory of gradient descent with $\eta = 0.01$

Effect of the step-size: example



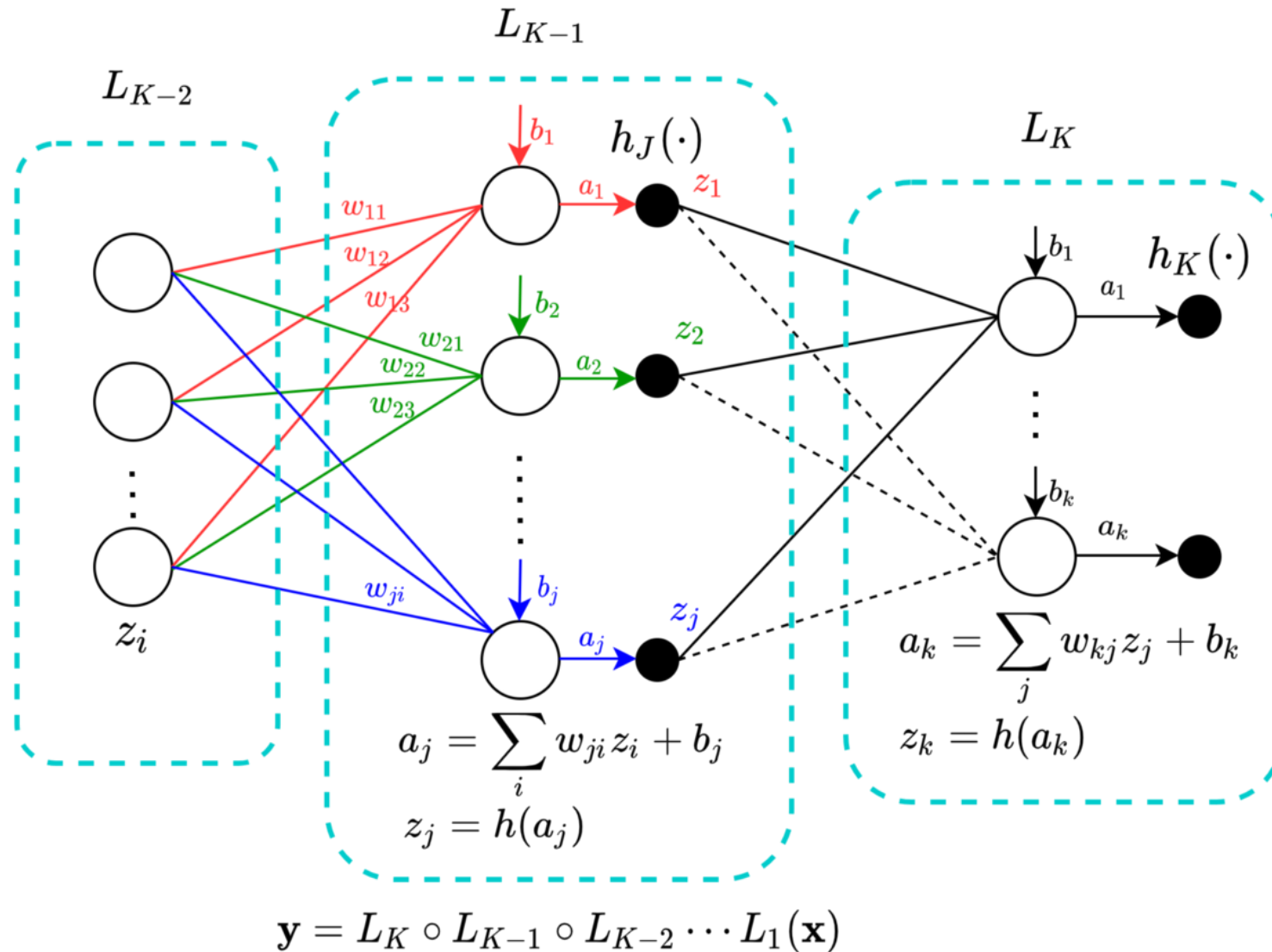
Trajectory of gradient descent with $\eta = 0.041$

Effect of the step-size: example



Trajectory of gradient descent with $\eta = 0.035$

Okay... how to apply GD to multiple layers?





Thank you – Any questions?



**Universiteit
Leiden**
The Netherlands

e.raponi@liacs.leidenuniv.nl

neural.computing@issc.leidenuniv.nl